KONERU LAKSHMAIAH EDUCATION FOUNDATION (KL UNIVERSITY)

[Department Name - edit here]

EduTrack

Data Structures and Algorithms-Based Academic Data Analysis System

Data Structures and Algorithms (DSA) Project Report

Submitted for External Review

Item	Details
Student name	Sravya [add surname / full name if required]
Registration number	[Registration number - edit here]
Department / Programme	[Department] , B.Tech (First Year)
University	KL University (Koneru Lakshmaiah Education Foundation)
Guide / Faculty name	[Guide name - edit here]
Academic year	[Academic year, e.g. 20XX - 20XX]

Language and platform: Java 21, console application

Certificate

This is to certify that the project report titled **"EduTrack: Data Structures and Algorithms-Based Academic Data Analysis System"** is a record of the project work carried out by **Sravya** (Registration No. [Registration number]), a B.Tech student of the Department of [Department Name], KL University (Koneru Lakshmaiah Education Foundation), during the academic year [Academic year], under the guidance of [Guide name].

The work has been prepared as part of the Data Structures and Algorithms course requirement.

This page is an editable template. It has not been signed or approved. Signatures and dates must be added by the concerned authorities.

Guide	**Head of the Department**	**External Examiner**
_____	_____	_____
Name: [Guide name]	Name: [HoD name]	Name: [Examiner name]
Signature and date	Signature and date	Signature and date

Institution seal: [place official seal here]

Acknowledgement

I would like to express my sincere gratitude to my guide, [Guide name], for the guidance, encouragement and valuable suggestions given throughout the development of this project.

I thank the Head of the Department of [Department Name] and the faculty members of KL University (Koneru Lakshmaiah Education Foundation) for providing the academic environment and the resources needed to complete this work. I am also grateful to my family and friends for their continuous support. Finally, I acknowledge the authors of the standard algorithm textbooks and the publicly documented student-performance dataset format that made this study possible.

Sravya

Abstract

Academic records are text-heavy, contain many attributes per student and are queried in many different ways. Looking up data with ad-hoc scans is hard to justify and hard to explain. EduTrack is a Java console application that treats a student-record dataset as a collection of data-processing problems and solves each one with a suitable, well-known algorithm written from scratch.

The system loads a CSV file of 649 student records with 33 attributes each (demographics, family background, study habits, absences and the grades G1, G2 and G3). A menu-driven interface connects the user to six algorithm modules. **Module 1** routes a query type to an algorithm family and tests primality with Miller-Rabin. **Module 2** searches the dataset text using Naive, Knuth-Morris-Pratt, Z, Rabin-Karp and Aho-Corasick matching, and builds a suffix array with a Kasai LCP array. **Module 3** applies dynamic programming: edit distance, sequence alignment, matrix-chain, optimal binary search tree, travelling salesman by bitmask, tree DP and sum-over-subsets. **Module 4** implements network flow (Ford-Fulkerson, Edmonds-Karp, Dinic), bipartite matching, min-cost max-flow and a deadline-scheduling reduction. **Module 5** demonstrates NP-completeness ideas: certificate checkers, two reductions, a 2-approximation for Vertex Cover, a greedy knapsack and a fixed-parameter search. **Module 6** covers randomised QuickSort, universal hashing, reservoir sampling and parallel work-span and scan primitives.

All algorithm engines avoid java.util collections and use custom containers. Every feature can be run from the menu, and a built-in benchmark cross-checks the modules. In the verified run, KMP located 317 occurrences of "services" and the six-module verification printed six OK lines. The project gives students and examiners a transparent, reproducible way to see how algorithm choice relates to problem type and complexity.

Keywords: data structures, algorithms, string matching, dynamic programming, network flow, NP-completeness, approximation, Java.

1. Introduction

1.1 About EduTrack

EduTrack is a console application written in Java 21. It reads student-record data from a CSV file and offers a menu of algorithm demonstrations that operate on that data and on small built-in sample inputs. The project is organised into a command-line layer (`edutrack.cli`), a support layer (`edutrack.core`) and six engine packages (`edutrack.module1` to `edutrack.module6`).

1.2 Background and motivation

Schools and universities store records such as demographics, study time, absences and grades. As the number of records and attributes grows, simple scanning becomes slow to reason about, and different questions need different tools: finding a keyword, finding repeated text, tolerating spelling mistakes, planning a route, assigning mentors, or modelling exam clashes. The DSA course teaches many techniques, but they are often studied in isolation. EduTrack places them side by side in one application so that each technique is tied to a realistic academic problem, can be run repeatedly and can be explained through its complexity.

1.3 Why DSA matters for academic data

- **Predictable cost:** an algorithm with a known bound (for example O(n + m) for KMP) behaves in an explainable way as data grows.
- **Right tool for the problem:** string matching, dynamic programming, flow and approximation each address a different kind of question.
- **Verifiability:** results such as match counts or flow values can be cross-checked between algorithms, which the built-in benchmark does.

2. Problem Statement

Academic records are large, text-heavy and queried in many different ways. Ad-hoc scans and hand-written lookups are slow to reason about and hard to justify. The project addresses four groups of problems, taken from the project presentation and confirmed in the source code:

Problem	Meaning in EduTrack	Where implemented
Search at scale	Locate keywords in the dataset text (649 records, 33 attributes) with exact line, column and offset	Module 2: Naive, KMP, Z, Rabin-Karp, Aho-Corasick
Repeated-text analysis	Find repeated or shared substrings without comparing every pair of positions	Module 2: Suffix Array, Kasai LCP, longest common substring
Inexact queries	Tolerate typing mistakes by measuring how close two strings are	Module 3: Levenshtein, Damerau-Levenshtein, alignment
Optimisation and conflicts	Plan tours, split intervals optimally, allocate flow, model exam conflicts	Modules 3, 4 and 5

3. Objectives

1. Map each academic problem to a suitable algorithm and justify the choice.
2. Implement the algorithms without java.util collections, using custom containers (`CustomIntArray`, `CustomIntQueue`, `CustomStringArray`, `CustomMatchList`).
3. Demonstrate results through verifiable, repeatable console runs from a single menu.
4. Compare approaches by their analytical complexity rather than by unsupported timings.
5. Provide a built-in benchmark option that cross-checks the six modules against each other.

Table of Contents

Certificate **2**
Acknowledgement **3**
Abstract **3**
1. Introduction **4**
2. Problem Statement 4
3. Objectives 4
4. Scope and Applications **6**
5. Existing and Proposed System 6
6. Requirements 6
7. System Architecture **7**
7.1 Overall workflow 7
7.2 Components in simple language 7
8. Project Structure and Dataset **8**
8.1 Project structure 8
8.2 Dataset description and verified statistics 8
9. Module-Wise Implementation **9**
9.1 Module 1: Foundational Engine (Query Routing, Miller-Rabin) 9
9.2 Module 2: String Algorithms 9
9.3 Module 3: Advanced Dynamic Programming 10
9.4 Module 4: Network Flow 11
9.5 Module 5: NP-Completeness and Approximation 12
9.6 Module 6: Randomised and Parallel Algorithms 13
10. Algorithm Explanations and Complexity Summary **14**
11. Execution and User Guide **15**
11.1 Running the project 15
11.2 Menu routes and main-menu screenshot 15
12. Execution Screenshots and Output Analysis **16**
12.1 String search and network flow 16
12.2 Module 5: reductions and approximation 17
12.3 Automated six-module verification 18
13. Testing and Validation **18**
14. Results and Discussion **19**
15. Limitations and Future Enhancements **19**
16. Conclusion **20**
17. References **20**

4. Scope and Applications

Scope. EduTrack covers six algorithm modules, dataset inspection (summary, ID lookup, preview) and an automated verification run. Input is taken from the keyboard through a menu; output is printed to the console. **Intended users** are first-year B.Tech students learning DSA and examiners or instructors reviewing how algorithms map to problems.

Practical applications: keyword and attribute lookup in text-form student records (Module 2); query correction for misspelt names or words (Module 3 edit distance); mentor assignment and task scheduling modelled as flow problems (Module 4); exam-conflict modelling with a vertex-cover approximation (Module 5).

Boundaries: it is a teaching and demonstration system, not a production student-information system; there is no graphical interface, database, network access or user login; several Module 3 to 6 demonstrations run on built-in sample inputs rather than the full dataset; no predictive modelling of grades is performed.

5. Existing and Proposed System

A basic approach would be to open the file and scan it by hand or with a simple loop for each question. Every new question needs a new hand-written scan whose cost is not stated, repeated-text and approximate-match questions are awkward with plain loops, and there is no built-in way to confirm that two methods agree. This section compares only a basic approach with EduTrack; no claims are made about other software.

Limitation of a basic approach	EduTrack response
Slow, unexplained scanning	Linear-time KMP and Z matching; Naive search kept as a baseline for comparison
Exact match only	Edit distance (Levenshtein, Damerau) measures closeness of strings
No view of repeated text	Suffix array and Kasai LCP find repeated and common substrings
Allocation done by trial	Max-flow, matching and min-cost flow solve assignment problems systematically
No way to confirm results	Menu option 8 cross-checks the six modules (for example KMP, Z and Rabin-Karp counts)

6. Requirements

Item	Requirement	Basis
Language	Java	All 15 source files are .java
JDK / JRE	JDK 21 to compile; Java 21 runtime to run	Shipped .class files are class-file version 65 (Java 21); compiled and run with OpenJDK 21
Operating system	Windows, Linux or macOS with a terminal	Pure Java, no native code; verified run was on Linux; Windows commands were not executed on Windows
IDE (optional)	Eclipse	Project contains .project, .classpath and .settings files from Eclipse
External libraries	None	Only the Java standard library is used

The project states no hardware requirement. The following are reasonable suggestions for a small console program with a 93 KB dataset, not measured values: any dual-core processor that runs Java 21; 2 GB RAM (4 GB recommended when using an IDE); about 50 MB free storage (source, classes and data are under 1 MB); a terminal window 110 columns wide or more.

7. System Architecture

7.1 Overall workflow

The user starts `EduTrackMain`, which first loads the dataset (it tries `data/student-mat.csv`, and if not found falls back to `student-por.csv`). It then runs a menu loop. Each choice calls static methods of an engine class directly and the result is printed to the console. As stated in the presentation, there is no separate service layer.

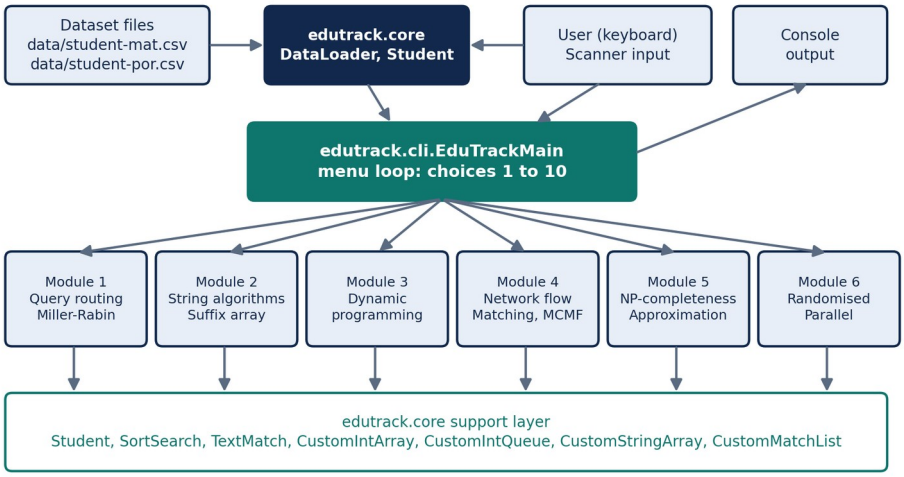


Figure 7.1: EduTrack system architecture (drawn for this report from the source code)

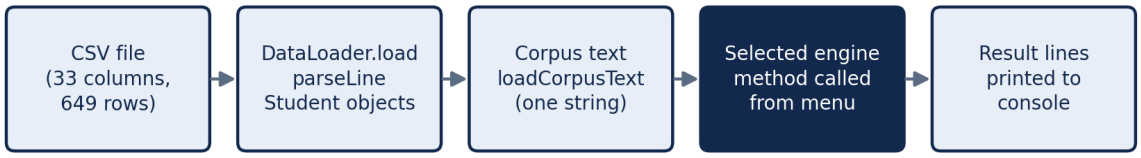


Figure 7.2: Data-processing workflow from CSV file to console output

7.2 Components in simple language

Component	What it does
Dataset files	Plain CSV files with one header line and one line per student
DataLoader / Student	Reads the file, splits each line (handling quotes), builds Student objects and reads the whole file as one text string for searching
EduTrackMain	Shows the menu, reads the choice and calls the correct engine
Module engines	Static methods that implement the algorithms of each module
Core containers	Custom array, queue, string-array and match-list classes used instead of java.util collections
Console output	Results printed as text, including counts, coordinates and tables

8. Project Structure and Dataset

8.1 Project structure

The structure was read directly from EduTrack.zip. The Java sources total 3,847 lines in 15 files.

```
EduTrack/   (.project, .classpath, .settings/ : Eclipse project files)
  data/   student-mat.csv, student-por.csv  (each: 1 header + 649 records)
  src/edutrack/
    cli/EduTrackMain.java
    core/ CustomIntArray, CustomIntQueue, CustomMatchList, CustomStringArray,
          DataLoader, SortSearch, Student, TextMatch
    module1/Module1Engine.java      module2/Module2StringEngine.java
    module3/Module3DPEngine.java    module4/Module4FlowEngine.java
    module5/Module5NPEngine.java    module6/Module6RandomParallelEngine.java
  bin/    compiled .class files (43) and a copy of student-por.csv
```

File	Lines	Purpose
cli/EduTrackMain.java	804	Main method, menu loop, sub-menus for every module, benchmark, dataset switching and path search
core/DataLoader, Student, SortSearch, TextMatch	75 / 76 / 68 / 20	CSV parser respecting quotes; one record (S001, S002, ...); bubble/merge sort, linear/binary search with counters; one match
module1/Module1Engine.java	199	Query routing table, Miller-Rabin primality, witness explanation
module2/Module2StringEngine.java	472	Naive, KMP, Z, Rabin-Karp, Aho-Corasick, suffix array, Kasai LCP
module3/Module3DPEngine.java	619	Edit distance, alignment, matrix-chain, OBST, TSP, Hamiltonian path, tree DP, SOS DP
module4/Module4FlowEngine.java	517	Ford-Fulkerson, Edmonds-Karp, Dinic, matching, MCMF, project selection, scheduling
module5/Module5NPEngine.java	350	Certificate checkers, reductions, approximations, FPT, taxonomy
module6/Module6RandomParallelEngine.java	366	Randomised QuickSort, universal hashing, reservoir sampling, Brent analysis, tree reduction, Blelloch scan

The source search found no java.util import in the core or module packages; java.util.Scanner is used only in EduTrackMain (the CLI), as stated in the presentation.

8.2 Dataset description and verified statistics

Two CSV files are included, each with a header of 33 column names followed by 649 records (verified by parsing both files in Python and by the program, which reports "Total Students Loaded: 649"). The 33 columns fall into five groups: identity and school (school, sex, age, address); family (famsize, Pstatus, Medu, Fedu, Mjob, Fjob, guardian, famrel); school and study (reason, traveltime, studytime, failures, schoolsup, famsup, paid, activities, nursery, higher, internet, romantic); lifestyle and health (freetime, goout, Dalc, Walc, health); and performance (absences, G1, G2, G3; grades 0 to 20). Descriptions follow the usual meaning of these standard attribute names; the files contain no data dictionary.

Loading: `EduTrackMain.loadSpecificDataset` tries `data/<file>`, `../data/<file>`, `<file>` and some fixed Windows paths; `DataLoader.load` skips lines whose field count differs from the header; `loadCorpusText` reads the whole file into one string that the string-matching modules search.

Statistic (from the file)	student-mat.csv	student-por.csv
Records / columns	649 / 33	649 / 33
School GP / MS; Sex F / M	423 / 226; 383 / 266	423 / 226; 383 / 266
Age range	15 to 22	15 to 22
Average G3; average absences; maximum absences	11.91; 3.66; 32	11.91; 3.66; 32
Text length (characters)	93,219	not computed

Limitations and a discrepancy: the two files differ in one place only (file line 374, field nursery: "no" in student-mat, "yes" in student-por). The dataset-switch menu text says student-mat.csv has 395 students, but the file has 649; this report uses the verified count. The loader skips malformed lines silently, does not validate ranges and reads a non-numeric value as 0. With no file found the program starts with zero students. The data describes students in a school study, not KL University students.

9. Module-Wise Implementation

The source code confirms six implemented modules. Each is summarised by purpose, concept, classes, input/output, complexity and use, followed by the key algorithm steps.

9.1 Module 1: Foundational Engine (Query Routing and Miller-Rabin)

Purpose	Show which algorithm family suits which kind of query, and test whether a large number is prime.
Concept	Lookup table (query class to algorithm family); Miller-Rabin test using modular exponentiation.
Classes	`Module1Engine`: getCanonTable, routeQueryDescription, millerRabin(long n, int k), powerMod, mulMod, explainMillerRabin. CLI: runModule1Menu (main menu option 2).
Input / output	Sub-menu 1: query type gives the mapped algorithm. Sub-menu 3: a number gives PRIME or COMPOSITE. Sub-menu 4 prints a witness trace.
Complexity	One base costs O(log² n) (O(log n) multiplications, each mulMod is O(log n)); k bases give O(k log² n). Space O(1).
Practical use	Routes a question to the right module; verifies large primes such as 1000000007.

Miller-Rabin as coded: reject n < 2 and even numbers (accept 2, 3); write n - 1 = d × 2^s with d odd; for each base a in {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37} compute x = a^d mod n; the base passes if x is 1 or n - 1, or if squaring up to s - 1 times reaches n - 1; one failing base means composite, otherwise prime. The menu message ">99.9999% confidence" is fixed text.

9.2 Module 2: String Algorithms

Purpose	Search the dataset text for keywords and analyse repeated text.
Concept	Naive, KMP, Z, Rabin-Karp (rolling hash) and Aho-Corasick matching; suffix array by prefix doubling; Kasai LCP.
Classes	`Module2StringEngine`: searchNaive, computeKmpPi, searchKMP, computeZArray, searchZ, searchRabinKarp, AhoCorasick, buildSuffixArray, buildKasaiLCP, longestRepeatedSubstring, longestCommonSubstring, mapOffsetsToCoordinates; uses TextMatch and CustomMatchList.
Input / output	Menu 3 option 1: a word (default "services") gives match count, time and the first six line/column/offset results. Option 5: a string gives the suffix array and LCP table. Options 2, 3, 4, 6, 7 are comparisons and demonstrations.
Complexity	Naive O(nm); KMP and Z O(n + m); Rabin-Karp expected O(n + m), worst O(nm); Aho-Corasick O(n + m + z); suffix array O(n log² n) (prefix doubling with merge sort, not SA-IS); Kasai O(n).
Practical use	Locating attributes or values in records; finding repeated profile text.

KMP: build the prefix table pi (longest proper prefix that is also a suffix of pattern[0..i]); scan the text once with a matched-count q; on a mismatch set q = pi[q - 1] so the text index never moves backwards; when q equals the pattern length record offset i - m + 1; `mapOffsetsToCoordinates` converts offsets to line and column.

Suffix array and Kasai: sort suffixes by first character, then for k = 1, 2, 4, ... merge-sort by the pair (rank[i], rank[i + k]) and recompute ranks until all ranks are distinct; Kasai fills the LCP array in O(n) by reusing the previous value minus one. For "banana" the maximum LCP is 3, giving "ana". **Rabin-Karp** compares rolling-hash values of pattern and window (moduli 101, or the double hash 1000000007 and 1000000009) and verifies every hash hit character by character; **Aho-Corasick** builds a trie with failure links to find several keywords in one pass.

9.3 Module 3: Advanced Dynamic Programming

Purpose	Solve optimisation and similarity problems by storing results of sub-problems.
Problem solved	Measure how close two strings are; align sequences; choose the cheapest order of matrix multiplications; plan a tour; analyse trees and subsets.
Concept	Table DP (edit distance, alignment), interval DP (matrix-chain, optimal BST), bitmask DP (TSP), tree DP, sum-over-subsets (SOS) DP.
Classes	`Module3DPEngine`: levenshteinMatrix, levenshteinDistance, damerauLevenshteinDistance, weightedEditDistance, calculateSimilarity, needlemanWunsch, smithWaterman, matrixChainOrder, optimalBST, solveTSP, findHamiltonianPath, computeTreeDP, sumOverSubsets, sumOverSupersets, explainWhenDPIsWrong.
Input / output	Menu 4, options 1 to 10, built-in sample inputs only. Example outputs: kitten/sitting = 3; matrix chain 4500; TSP cost 101.
Complexity	Edit distance and alignment O(nm); matrix-chain and OBST $O(n^3)$; TSP bitmask $O(2^n \times n^2)$; tree DP O(n); SOS DP O(k × 2^k) for k bits.
Practical use	Query correction (typos), tour planning, understanding when DP is or is not the right tool.

Edit distance step by step

1. Create a table D of size (n + 1) × (m + 1) with D[i][0] = i and D[0][j] = j.
2. For each cell take the minimum of delete (D[i-1][j] + 1), insert (D[i][j-1] + 1) and replace or keep (D[i-1][j-1] + 0 or 1).
3. Damerau-Levenshtein adds one more option: swapping two adjacent characters at cost 1.
4. The answer is D[n][m]. Example: "kitten" to "sitting" gives 3 (replace k by s, replace e by i, insert g); "ca" to "ac" gives 1 with Damerau.

TSP bitmask DP step by step

1. A bitmask stands for the set of visited nodes; dp[mask][u] is the cheapest way to start at node 0, visit exactly the nodes in mask and end at u.
2. Start with dp[1][0] = 0. For every mask containing node 0 and every end node u, try moving to each unvisited node v.
3. The answer is the minimum over u of dp[full][u] + dist[u][0]; a parent array rebuilds the tour.
4. For the built-in 6-node matrix the program prints cost 101 and path 0-5-4-3-2-1-0 (confirmed by a Python brute force over all 120 tours).

Other Module 3 techniques

Matrix-chain uses interval DP over split points; for dimensions 10×30, 30×5, 5×60 the best order costs 10×30×5 + 10×5×60 = 4500 multiplications. **Needleman-Wunsch and Smith-Waterman** fill an alignment table for global and local similarity. **Tree DP** combines child results bottom-up in O(n), and **SOS DP** accumulates subset sums over all bitmasks. `explainWhenDPIsWrong` prints a note on problems where DP does not apply (no overlapping sub-problems or no optimal substructure).

9.4 Module 4: Network Flow

Purpose	Model allocation and assignment problems as flow in a network.
Problem solved	Find the maximum flow from a source to a sink; assign students to mentors; schedule tasks before deadlines; choose projects with prerequisites.
Concept	Augmenting paths (Ford-Fulkerson with DFS, Edmonds-Karp with BFS), level graph and blocking flow (Dinic), max-flow min-cut, bipartite matching by reduction to flow, successive shortest paths for min-cost flow.
Classes	`Module4FlowEngine`: fordFulkerson, edmondsKarp, dinicMaxFlow, solveBipartiteMatching, solveMinCostMaxFlow (uses FlowNetwork, FlowEdge), solveProjectSelection, scheduleTasksWithDeadlines; uses CustomIntQueue.
Input / output	Menu 5, options 1 to 6, built-in sample networks. Option 1 prints a table of the three solvers: max flow 23 each, with 6, 3 and 2 augmenting steps.
Complexity	Edmonds-Karp $O(V E^2)$; Dinic general bound $O(V^2 E)$; Ford-Fulkerson $O(E \times \lvert f \rvert)$ for integer capacities.
Practical use	Mentor assignment, task allocation under capacity, and project selection.

Edmonds-Karp step by step

1. Build a residual capacity matrix from the capacities.
2. Use BFS to find the shortest source-to-sink path that still has spare capacity.
3. Push the bottleneck (smallest spare capacity on the path) along the path; update forward and backward residuals.
4. Repeat until BFS finds no path. The total pushed is the maximum flow.

Bipartite matching and other flow models

Bipartite matching adds a source linked to every left node (capacity 1), a sink linked from every right node (capacity 1) and a capacity-1 edge for each allowed pair; the max flow equals the matching size. This models assigning students to mentors. **Min-cost max-flow** repeatedly augments along the cheapest residual path (successive shortest paths). **Project selection** reduces to a minimum cut, and **deadline scheduling** is expressed as a flow from tasks to time slots.

Why three solvers? Ford-Fulkerson (DFS), Edmonds-Karp (BFS) and Dinic (level graph with blocking flow) must agree on the same network. They return 23 each with 6, 3 and 2 augmenting steps, which cross-checks the implementations and shows how the choice of augmenting strategy changes the work done (see Figure 12.2).

9.5 Module 5: NP-Completeness and Approximation

Purpose	Demonstrate hard problems, verification, reductions and approximation.
Problem solved	Some problems (such as minimum vertex cover) have no known polynomial-time exact algorithm. The module shows how to check an answer quickly, relate problems, and get a provably close answer.
Classes	`Module5NPEngine`: verifyVertexCover, verifySubsetSum, verify3SAT, reduce3SatToClique, reduceCliqueToIndependentSet, approximateVertexCover, approximateKnapsack, solveFPTVertexCover, getComplexityTaxonomy.
Input / output	Menu 6, options 1 to 5, built-in sample graphs and formulas. Option 3 prints |M| = 3, |C| = 6 for a 6-student conflict graph.
Complexity	Certificate checks run in polynomial time; the 2-approximation with an adjacency matrix is $O(V^2)$; the FPT search branches on an edge, so its time depends on the parameter k.
Practical use	Exam-conflict modelling: choosing a small set of students or proctors that touches every conflict.

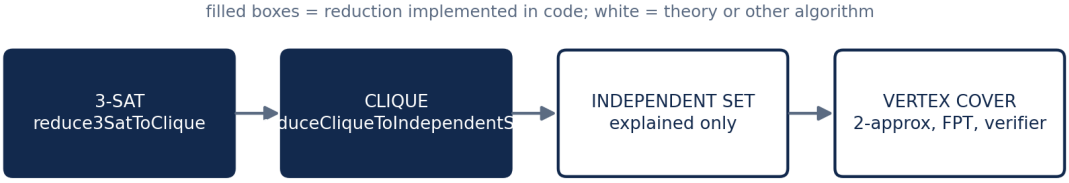


Figure 9.1: Module 5 reduction chain: what is coded and what is theory only

Concept	Meaning	In the project
P	Problems solvable in polynomial time	Explained; certificate verification runs in polynomial time
NP	A proposed solution can be checked in polynomial time	verifyVertexCover, verifySubsetSum, verify3SAT
NP-complete	In NP, and every NP problem reduces to it	3-SAT, CLIQUE, VERTEX COVER named in the reduction chain
NP-hard	At least as hard as NP-complete problems; optimisation versions are NP-hard	Explained in text output
Polynomial-time reduction A ≤p B	A fast conversion showing B is at least as hard as A	Coded: 3-SAT to CLIQUE, CLIQUE to INDEPENDENT SET (complement graph). Theory only: INDEPENDENT SET to VERTEX COVER, SUBSET-SUM to KNAPSACK, HAM-CYCLE to TSP
Vertex Cover	Smallest vertex set touching every edge (NP-complete decision version)	Certificate checker, 2-approximation and FPT search
Approximation ratio	Solution within a factor r of the optimum	Coded: Vertex Cover 2-approximation, greedy Knapsack. PTAS, FPTAS, APX printed as a taxonomy table only (option 5)

Vertex Cover 2-approximation: copy the graph and start with an empty cover C; scan vertex pairs (u, v) and, whenever an edge (u, v) is still present, count it in the matching M, add u and v to C and remove every edge touching u or v. At the end $|C| = 2|M|$. The matched edges share no vertex, so any cover needs at least $|M|$ vertices while C has $2|M|$; therefore $|C| \le 2 \times OPT$. The code does not compute OPT itself.

9.6 Module 6: Randomised and Parallel Algorithms

Purpose	Show how randomness and parallel thinking improve algorithms.
Problem solved	Avoid worst-case input for QuickSort; spread keys evenly in hash buckets; sample from a stream; analyse parallel speed-up.
Concept	Randomised QuickSort, universal hashing, reservoir sampling, work-span analysis with Brent's theorem, tree reduction and Blelloch scan (simulated).
Classes	`Module6RandomParallelEngine`: getParadigms, benchmarkQuickSort, testUniversalHashing, reservoirSample, analyzeBrentsTheorem, parallelTreeReduce, blellochParallelScan; its own generator setSeed, nextRandomInt.
Input / output	Menu 7, options 1 to 6, built-in inputs. Option 2 sorts a pre-sorted array of 150 scores with first-element and random pivots and prints comparison counts.
Complexity	First-pivot QuickSort on sorted input $O(n^2)$; randomised QuickSort expected O(n log n); reservoir sampling O(n); tree reduction and Blelloch scan O(n) work with logarithmic span in the parallel model.
Practical use	Robust sorting of marks, even distribution of student IDs in hash tables, sampling streaming submissions.

Randomised QuickSort idea

A QuickSort that always picks the first element as pivot makes n(n-1)/2 comparisons on already-sorted data; for n = 150 that is 11,175. The randomised version picks a random pivot (seeded from the system clock), so no fixed input is always bad. The program prints both comparison counts and the theoretical 2 n ln n estimate. The parallel items (Brent analysis, tree reduction, Blelloch scan) are simulations computed in a single thread; they illustrate the algorithms and are not multi-threaded speed measurements.

Summary of the six modules

Module	Menu	Engine class	Lines	Key algorithms
1 Foundational Engine	2	Module1Engine	199	Query routing, Miller-Rabin
2 String Algorithms	3	Module2StringEngine	472	Naive, KMP, Z, Rabin-Karp, Aho-Corasick, suffix array, Kasai LCP
3 Advanced DP	4	Module3DPEngine	619	Edit distance, alignment, matrix-chain, OBST, TSP, tree DP, SOS
4 Network Flow	5	Module4FlowEngine	517	Ford-Fulkerson, Edmonds-Karp, Dinic, matching, min-cost flow
5 NP-Completeness and Approximation	6	Module5NPEngine	350	Certificates, reductions, 2-approximation, knapsack, FPT
6 Randomised and Parallel	7	Module6RandomParallelEngine	366	Randomised QuickSort, universal hashing, reservoir sampling, scan

Menu 1 covers dataset analytics and record inspection, menu 8 runs the six-module benchmark, menu 9 switches the dataset and menu 10 exits.

10. Algorithm Explanations and Complexity Summary

The pseudocode follows the structure of the actual Java methods named in each heading.

10.1 KMP search (computeKmpPi, searchKMP)

```
function computePi(pat):
  pi[0] = 0; k = 0
  for i = 1 to m-1:
    while k > 0 and pat[k] != pat[i]: k = pi[k-1]
    if pat[k] == pat[i]: k = k + 1
    pi[i] = k
  return pi
```

```
function searchKMP(text, pat):
  pi = computePi(pat); q = 0
  for i = 0 to n-1:
    while q > 0 and pat[q] != text[i]: q = pi[q-1]
    if pat[q] == text[i]: q = q + 1
    if q == m: record (i - m + 1); q = pi[q-1]
```

Example. Pattern "aba" has pi = [0, 0, 1]. In the text "ababa", q reaches 3 at index 2 (match at offset 0), falls back to pi[2] = 1 and reaches 3 again at index 4 (match at offset 2); the text pointer never moved backwards.

10.2 Recurrences and Vertex Cover approximation

```
Edit distance: D[i][j] = min(D[i-1][j]+1, D[i][j-1]+1, D[i-1][j-1] + (s1[i-1] != s2[j-1]))
TSP bitmask:   dp[mask|(1<<v)][v] = min(dp[mask|(1<<v)][v], dp[mask][u] + dist[u][v])
               answer = min over u of dp[full][u] + dist[u][0]
Edmonds-Karp:  while BFS finds a path s to t: b = bottleneck; push b; flow += b
Vertex Cover:  for u < v: if rem[u][v]: matching++; add u, v to C; delete edges at u, v
               return C, matching        // |C| = 2 * matching <= 2 * OPT
```

10.3 Complexity summary

Algorithm (module)	Time	Algorithm (module)	Time
Naive search (2)	O(nm)	Edit distance, alignment (3)	O(nm)
KMP, Z search (2)	O(n + m)	Matrix-chain, optimal BST (3)	$O(n^3)$
Rabin-Karp (2)	expected O(n + m); worst O(nm)	TSP bitmask DP (3)	$O(2^n \times n^2)$
Aho-Corasick (2)	O(n + m + z)	Tree DP (3); SOS DP (3)	O(n); O(k × 2^k)
Suffix array, prefix doubling (2)	$O(n \log^2 n)$	Edmonds-Karp (4)	$O(V E^2)$
Kasai LCP (2)	O(n)	Dinic (4); Ford-Fulkerson (4)	$O(V^2 E)$; O(E × |f|)
Miller-Rabin, k bases (1)	$O(k \log^2 n)$	Vertex Cover 2-approximation (5)	$O(V^2)$, ratio 2
First-pivot QuickSort, sorted input (6)	$O(n^2)$	Randomised QuickSort (6); reservoir (6)	expected O(n log n); O(n)

Comparisons between algorithms in this report rest on analytical complexity and on deterministic comparison counts. Microsecond timings printed by the program come from single runs on one machine, include Java virtual machine warm-up and are not used for performance claims.

11. Execution and User Guide

11.1 Running the project

Start the program from the project root (the folder containing bin, src and data). `EduTrackMain` looks for `data/student-mat.csv` relative to the current folder (then `../data/`, then the bare file name); if it starts elsewhere it may not find the dataset and will report zero students. Compiled classes already exist in bin, so the javac line is optional. The same compile command on Linux with OpenJDK 21 produced the same 43 class files as the shipped bin folder; it was not executed on Windows.

```
:: Windows (Command Prompt), from the project root
javac -d bin -sourcepath src src\edutrack\cli\EduTrackMain.java
java -cp bin edutrack.cli.EduTrackMain
# Linux / macOS, from the project root
javac -d bin -sourcepath src src/edutrack/cli/EduTrackMain.java
java -cp bin edutrack.cli.EduTrackMain
```

Eclipse: extract the ZIP, use File > Import > Existing Projects into Workspace, then right-click `edutrack.cli.EduTrackMain` and choose Run As > Java Application. **Menu use:** type the number of a main-menu option and press Enter, then a sub-menu number; pressing Enter on an empty line uses the default where one is shown; choose 10 to exit.

11.2 Menu routes and main-menu screenshot

Goal	Menu	Input	What to look for
Dataset summary / student lookup	1 > 1 / 1 > 2	none / S015	649 students, average G3 11.91, absences 3.66 / record with comparison counts
Primality test	2 > 3	1000000007	CERTIFIED PRIME
Keyword search / suffix array	3 > 1 / 3 > 5	services / banana	317 matches with line, column, offset / SA and LCP table, max LCP 3
Edit distance / matrix chain / TSP	4 > 1 / 4 > 4 / 4 > 6	none	3 and 1 / 4500 / 101
Max-flow solvers	5 > 1	none	23 for all three solvers
Reductions / vertex cover	6 > 2 / 6 > 3	none	3-SAT to CLIQUE and complement graph / |M| = 3, |C| = 6
Randomised QuickSort / verification	7 > 2 / 8	none	11175 vs a much smaller random count / six [OK] lines
Switch dataset	9 > 1 or 2	1 or 2	Dataset reloaded; 649 students

Screenshot capture. The shipped classes were run with OpenJDK 21 in a real pseudo-terminal; the menu inputs were typed as a user would and the recorded terminal text was drawn onto images with a monospaced font. They are real captured sessions, not mockups. The repeated main menu is cropped out, the banner path belongs to the verification environment, and timings vary between runs.

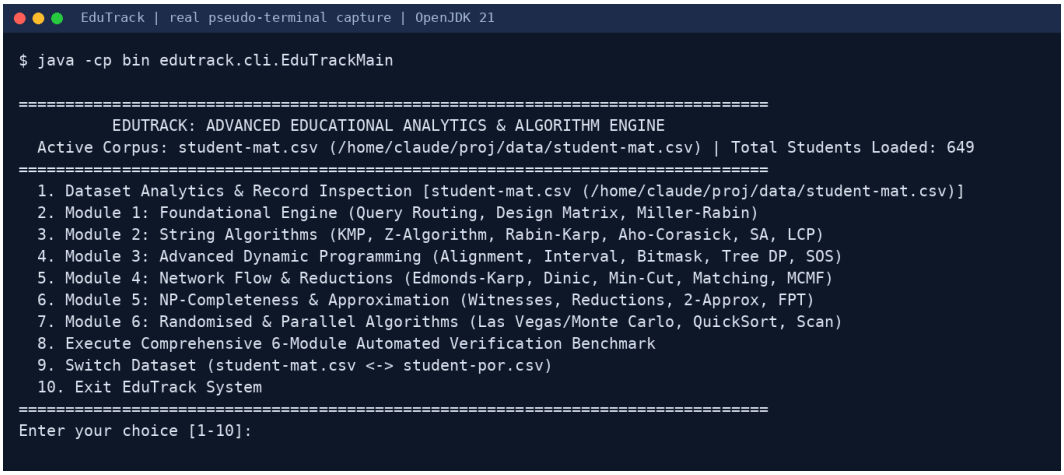


Figure 11.1: EduTrack launch and main menu

Entered: the launch command. **Result:** the dataset loaded (student-mat.csv, 649 students), the banner and ten-option menu were printed, and options 2 to 7 map to Modules 1 to 6 with option 8 as the benchmark. **Verifies:** dataset loading (T01) and the menu structure of Chapter 7.

12. Execution Screenshots and Output Analysis

12.1 String search and network flow

```
EduTrack | real pseudo-terminal capture | OpenJDK 21

Enter your choice [1-10]: 3

--- [EduTrack Module 2: Exact String Algorithms Suite] ---
1. Knuth-Morris-Pratt (KMP) Pattern Search with Line/Col Coordinates
2. Comparative Benchmark (Naive vs KMP vs Z-Algorithm vs Rabin-Karp)
3. Rabin-Karp Spurious Collision Demonstration (Mod 101 vs Double Hash)
4. Aho-Corasick Multi-Pattern Keyword Automaton
5. Suffix Array & Kasai LCP Array Construction
6. Longest Common Substring / Profile Similarity (Suffix Structures)
7. Suffix Structures Theoretical Comparison (SA vs Tree vs Automaton)
Select Option [1-7]: 1
Enter attribute/word to locate (default 'services'): services

[+] Located 317 matches for "services" in 19556 us.
    [Line: 5, Col: 41, Absolute Offset: 694] -> "services"
    [Line: 7, Col: 32, Absolute Offset: 968] -> "services"
    [Line: 10, Col: 32, Absolute Offset: 1396] -> "services"
    [Line: 13, Col: 32, Absolute Offset: 1826] -> "services"
    [Line: 14, Col: 41, Absolute Offset: 1984] -> "services"
    [Line: 18, Col: 32, Absolute Offset: 2543] -> "services"
    ... (311 more coordinates recorded)
```

Figure 12.1: Module 2: KMP keyword search with coordinates (Menu 3 > 1)

Entered: 3, 1, then `services`. **Result:** the whole dataset text was searched with KMP and offsets were converted to line and column. 317 occurrences were found; the first is at line 5, column 41, absolute offset 694. The count equals Python str.count on the file and the first offset equals Python find. **Verifies:** searchKMP and mapOffsetsToCoordinates (T07, T08).

```
EduTrack | real pseudo-terminal capture | OpenJDK 21

Enter your choice [1-10]: 5

--- [EduTrack Module 4: Network Flow Suite & Reductions] ---
1. Compare Max-Flow Solvers (Ford-Fulkerson vs Edmonds-Karp vs Dinic's)
2. Min-Cut Dual Analysis (Max-Flow Min-Cut Theorem & Cut Edges)
3. Maximum Bipartite Matching (Student-to-Mentor Assignment & Konig's Theorem)
4. Min-Cost Max-Flow (MCMF via Successive Shortest Path)
5. Practical Reduction: Project Selection with Prerequisites (Min-Cut)
6. Practical Reduction: Task/Course Scheduling with Deadlines
Select Option [1-6]: 1

================================================================================
  Flow Algorithm           Complexity       Max Flow  Augmenting Steps
================================================================================
  1. Ford-Fulkerson (DFS)   O(E * |f|)       23        6
  2. Edmonds-Karp (BFS)     O(V * E^2)       23        3
  3. Dinic's Algorithm      O(V^2 * E)       23        2
================================================================================
```

Figure 12.2: Module 4: comparison of max-flow solvers (Menu 5 > 1)

Entered: 5, then 1. **Result:** Ford-Fulkerson, Edmonds-Karp and Dinic ran on the same sample network. All three give maximum flow 23, needing 6, 3 and 2 augmenting steps respectively; the same answer by different routes cross-checks the implementations. **Verifies:** fordFulkerson, edmondsKarp, dinicMaxFlow (T16).

12.2 Module 5: reductions and approximation

```
EduTrack | real pseudo-terminal capture | OpenJDK 21

Enter your choice [1-10]: 6

--- [EduTrack Module 5: NP-Completeness & Approximation] ---
1. Polynomial-Time Certificate Verification (Witness Validation)
2. The Reduction Zoo (3-SAT -> CLIQUE -> INDEPENDENT-SET -> VERTEX-COVER)
3. 2-Approximation Algorithms (Vertex Cover via Maximal Matching & Knapsack)
4. Parameterized Complexity & FPT (Bounded Search Tree for Vertex Cover)
5. Complexity Class Taxonomy (PTAS vs FPTAS vs APX-Hard Framework)
Select Option [1-5]: 2

>> 3-SAT -> CLIQUE Polynomial Reduction:
   Formula: (x1 v x2 v x3) ^ (~x1 v ~x2 v x3)
   Constructed Graph: 6 vertices (3 per clause)
   Target Clique Size: 2 (Formula is SAT iff graph has 2-clique)

>> CLIQUE -> INDEPENDENT-SET -> VERTEX-COVER Equivalence:
   Complement graph constructed with 6 vertices.
   Educational Mapping: Finding maximum conflict-free student cohorts (Independent Set)
   is mathematically dual to finding minimum proctors to cover all conflicts (Vertex Cover).
```

Figure 12.3: Module 5: reduction sequence (Menu 6 > 2)

Entered: 6, then 2. **Result:** a graph was built from a sample 3-SAT formula and then its complement graph. The formula (x1 or x2 or x3) and (not x1 or not x2 or x3) gives a graph of 6 vertices (3 per clause) with target clique size 2: the formula is satisfiable exactly when a 2-clique exists. The complement graph (also 6 vertices) represents the CLIQUE to INDEPENDENT SET step; the link to VERTEX COVER is explained in text only. **Verifies:** reduce3SatToClique and reduceCliqueToIndependentSet.

```
EduTrack | real pseudo-terminal capture | OpenJDK 21

Enter your choice [1-10]: 6

--- [EduTrack Module 5: NP-Completeness & Approximation] ---
1. Polynomial-Time Certificate Verification (Witness Validation)
2. The Reduction Zoo (3-SAT -> CLIQUE -> INDEPENDENT-SET -> VERTEX-COVER)
3. 2-Approximation Algorithms (Vertex Cover via Maximal Matching & Knapsack)
4. Parameterized Complexity & FPT (Bounded Search Tree for Vertex Cover)
5. Complexity Class Taxonomy (PTAS vs FPTAS vs APX-Hard Framework)
Select Option [1-5]: 3

>> Vertex Cover 2-Approximation via Maximal Matching:
   6-Student Exam Collaboration Conflict Graph:
   Maximal Matching Size (|M|): 3
   Constructed Vertex Cover Size (|C| = 2*|M|): 6
   Selected Proctor Node Indices: [0, 1, 2, 3, 4, 5]
   Ratio Guarantee: |C| <= 2 * OPT (Proved because any valid cover requires >= |M| vertices).

>> Greedy Knapsack 0.5-OPT Approximation (Project Credit Allocation):
   Values: [60, 100, 120], Weights: [10, 20, 30], Capacity: 50
   Selected Items: [0, 1]
   Total Weight: 30 / 50
   Achieved Value: 160 (Guaranteed >= 0.5 * OPT)
```

Figure 12.4: Module 5: approximation algorithms (Menu 6 > 3)

Entered: 6, then 3. **Result:** the maximal-matching Vertex Cover approximation ran on a 6-student conflict graph and the greedy knapsack on a small instance. The matching has 3 edges, so the cover has 6 vertices, at most twice the optimum. The knapsack picks items 0 and 1 for value 160 (the best possible here is 220, found by hand). **Verifies:** approximateVertexCover (T17) and approximateKnapsack.

12.3 Automated six-module verification

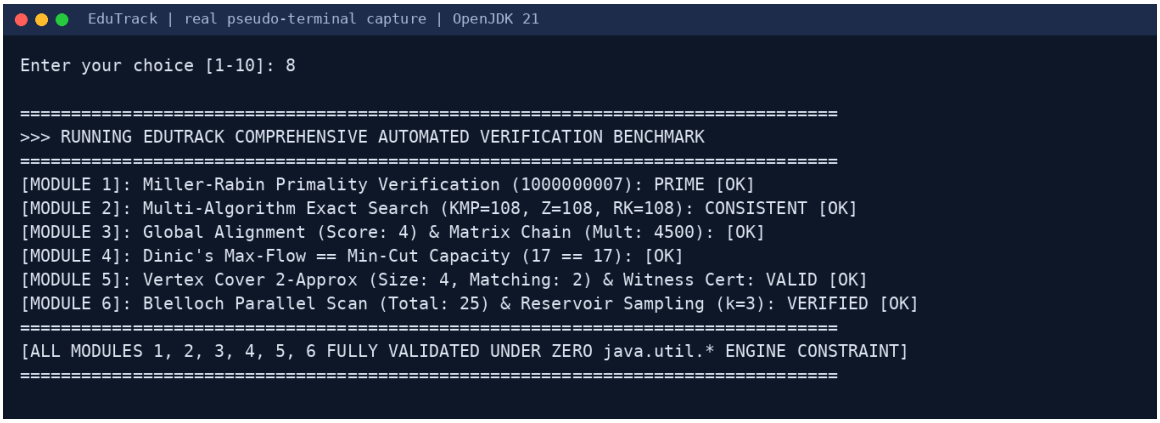


Figure 12.5: Automated six-module verification (Menu 8)

Entered: 8. **Result:** one check per module, six OK lines: Miller-Rabin accepts 1000000007; KMP, Z and Rabin-Karp all find 108 matches of "teacher"; alignment score 4 and matrix chain 4500; Dinic max-flow equals min-cut (17 = 17); a 2-approximation cover with a valid witness; Blelloch scan total 25 and reservoir sampling with k = 3. The "zero java.util" line is fixed text printed by the program, not a runtime check. **Verifies:** the cross-module verification (T19).

13. Testing and Validation

Tests were executed against the shipped bin classes by feeding menu inputs through standard input and checking the output text. Expected values came from independent sources where possible: Python for counts, averages and a brute-force TSP over all 120 tours, and textbook values or simple arithmetic for the others. Timings were not pass criteria.

ID	Feature / input	Expected	Actual	Status
T01	Dataset loading (launch)	649 students loaded	649 students loaded	PASS
T02-03	Dataset summary, Menu 1 > 1	Average G3 11.91; absences 3.66	11.91 / 20.0; 3.66 days	PASS
T04-05	Lookup S015: linear / binary	15 comparisons; at most 10	15 comparisons; 7	PASS
T06	Lookup invalid ID S999	Not-found message	Student with ID 'S999' not found	PASS
T07-08	KMP "services"	317 matches; line 5, col 41, offset 694	Located 317; [5, 41, 694]	PASS
T09	Suffix array, "banana"	SA = 5 3 1 0 4 2; max LCP 3	SA rows matched	PASS
T10-11	Levenshtein / Damerau	kitten/sitting = 3; ca/ac = 1	3; 1	PASS
T12	Matrix-chain, Menu 4 > 4	4500	4500	PASS
T13	TSP vs brute force, Menu 4 > 6	101 (120 tours in Python)	Minimum Tour Cost: 101	PASS
T14-15	Miller-Rabin 1000000007 / 561	Prime / composite (Carmichael)	CERTIFIED PRIME / COMPOSITE	PASS
T16	Max-flow solvers agree	Same flow from all three	23, 23, 23	PASS
T17	Vertex-cover 2-approximation	|M| = 3, |C| = 6	|M| = 3, |C| = 6	PASS
T18	QuickSort adversary (sorted 150)	150*149/2 = 11175	11175 comparisons	PASS
T19	Automated benchmark, Menu 8	Six [OK] lines	6 [OK] lines	PASS
T20	Switch dataset, Menu 9 > 2	student-por.csv, 649 students	Loaded, 649 students	PASS
T21	Invalid menu choice "abc"	Invalid-selection message	Invalid selection (1 to 10)	PASS
T22	Invalid number in Miller-Rabin, "abc"	Friendly error, back to menu	NumberFormatException (Long.parseLong, EduTrackMain line 160)	FAIL

Result: 22 tests were executed; 21 passed and 1 failed. T22 is a real robustness defect: a non-numeric entry for the Miller-Rabin number stops the program with an unhandled exception. It is reported in Chapter 15 and was not fixed, because the project code was not to be changed.

14. Results and Discussion

The runs show that EduTrack can load a 649-record dataset, search it and demonstrate algorithms from all six modules with outputs that match independent calculations.

- **Search:** KMP found 317 occurrences of "services", equal to Python's count, with coordinates for each match; in the benchmark KMP, Z and Rabin-Karp agreed on 108 matches of "teacher".
- **Dynamic programming:** edit distance, matrix-chain and TSP results matched textbook or brute-force values.
- **Flow:** three max-flow algorithms returned the same value (23) with different numbers of augmenting steps.
- **Approximation:** the Vertex Cover result obeyed $|C| = 2|M|$, consistent with the 2-approximation argument.
- **Randomisation:** randomised QuickSort used far fewer comparisons than the first-pivot version on sorted input (1,075 versus 11,175 in one run).

About measurements. Some outputs include microsecond timings (for example 31,251 us for the KMP search in one run). They come from single runs on one machine, include JVM warm-up and are not used for performance claims.

Presentation versus code. The presentation and the code agree on the six modules, menu routes and sample results (317, 4500, 101, $|M| = 3$, $|C| = 6$ were reproduced). The one mismatch is the dataset-switch text (395 students) against the file and presentation (649). Module 5 reductions listed as theory only are indeed not coded: only 3-SAT to CLIQUE and CLIQUE to INDEPENDENT SET are constructed.

Not executed: the program was not run on Windows; individual menu options other than those in Table 13 were not tested one by one (the benchmark, option 8, exercises one check per module); no performance benchmarking across data sizes was done.

15. Limitations and Future Enhancements

15.1 Limitations

- Modules 3 to 6 run on built-in sample inputs; only a few options accept typed input (search word, suffix-array string, primality candidate, student ID).
- Invalid numeric input in the Miller-Rabin option stops the program with an exception (T22); end of input at the main prompt also raises an exception.
- The suffix array uses prefix doubling with merge sort, $O(n \log^2 n)$, not a linear-time construction.
- Only two reductions are coded; INDEPENDENT SET to VERTEX COVER, SUBSET-SUM to KNAPSACK and HAM-CYCLE to TSP are explained only.
- The two dataset files are nearly identical, the switch-menu text claims 395 students for student-mat.csv, search paths include fixed Windows paths from the author's machine, and a missing file leads to an empty dataset.
- Console interface only, no automated unit-test suite; parallel items are simulated in one thread; the Miller-Rabin "confidence" message is fixed text; performance was not benchmarked across input sizes.

15.2 Future enhancements (proposals only)

- Accept user-supplied inputs for the dynamic programming and NP demonstrations, and add input validation with friendly error messages.
- Add a linear-time suffix array construction such as SA-IS, and implement the remaining reductions as executable code.
- Add unit tests and timed benchmarks on both datasets with repeated runs.
- Correct the dataset description text, remove hard-coded machine paths, and provide a simple graphical or web interface.

16. Conclusion

EduTrack applies appropriate data structures and algorithms to academic information-processing problems. Its six modules cover string matching and suffix structures, dynamic programming, network flow, NP-completeness with approximation, and randomised and parallel techniques, all written from scratch with custom containers.

Execution on OpenJDK 21 reproduced the results given in the project presentation, and 21 of 22 executed tests passed, with one input-validation defect identified. The project strengthened understanding of how algorithm choice follows problem type, how complexity bounds explain behaviour, and how results can be verified by cross-checking independent methods. It is useful as a teaching demonstration and a base for further work.

17. References

Only well-known standard sources are listed. No web addresses are given where the exact address could not be verified.

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press (edition as available in the course library).
2. D. E. Knuth, J. H. Morris and V. R. Pratt, "Fast pattern matching in strings", SIAM Journal on Computing, 1977.
3. R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms", IBM Journal of Research and Development, 1987.
4. A. V. Aho and M. J. Corasick, "Efficient string matching: an aid to bibliographic search", Communications of the ACM, 1975.
5. U. Manber and G. Myers, "Suffix arrays: a new method for on-line string searches", SIAM Journal on Computing, 1993.
6. T. Kasai et al., "Linear-time longest-common-prefix computation in suffix arrays and its applications", CPM 2001.
7. V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals", Soviet Physics Doklady, 1966.
8. J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems", Journal of the ACM, 1972.
9. E. A. Dinic, "Algorithm for solution of a problem of maximum flow in a network with power estimation", Soviet Math. Doklady, 1970.
10. M. R. Garey and D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman, 1979.
11. G. L. Miller, "Riemann's hypothesis and tests for primality", 1976; M. O. Rabin, "Probabilistic algorithm for testing primality", Journal of Number Theory, 1980.
12. Oracle, Java Platform, Standard Edition documentation (Java SE 21 API Specification).
13. Dataset: the student-record files student-mat.csv and student-por.csv supplied with the project (33 attributes; 649 records each). The original source of these files was not stated in the supplied material.
14. Project presentation: "EduTrack: Applying Data Structures and Algorithms to Academic Information Processing", DSA Project Review-1, supplied by the student.